

Paged Attention 与 Ring KV Cache

目 录

1 引言：KV Cache 的显存浪费	2
2 Paged Attention：按需分块	2
2.1 核心理念	2
2.2 Block Size 的权衡	2
2.3 功能正确版本	3
2.4 优化版本	3
3 Ring KV Cache：滑动窗口的天然适配	3
3.1 滑动窗口的特性	3
3.2 环形缓冲区	3
3.3 显存从 $O(S)$ 降到 $O(\min(S, W))$	4
3.4 混合注意力层的 KV 因子	4
4 实战：算一算省了多少	4
4.1 Ring KV 的显存节省	4
4.2 Paged Attention 的碎片对比	5
5 框架中的 KV Cache 实现	5
6 本章你将学会	5
7 要点速查	6
8 小结	6

1 引言：KV Cache 的显存浪费

LLM 推理时，KV Cache 是显存占用的主要来源之一。在静态分配策略下，框架按最大 batch size B 和最大序列长度 S 预分配 KV Cache：

$$M_{kv} = 2BSNH_{kv}D_h \cdot \frac{y}{8} \text{ bytes} \quad (1.1)$$

其中 $y = 16$ 为激活值位宽。这套公式假设每个请求都会用到最大长度 S 的 KV Cache，但现实中请求的生成长度差异极大。一个 prompt 长度 100、只生成 20 个 token 的请求，却占用了 $S = 2048$ 的 KV 槽位，剩余 1928 个槽位全部闲置。

更严重的是，Gemma4-12B 采用混合注意力：大部分层使用滑动窗口注意力，窗口大小 $w = 1024$ ，超过窗口的 token 的 KV 根本不会被访问。但静态分配仍然为每个请求保存了全长度的 KV，白白浪费了显存。

直觉 想象一家酒店为每位客人都预留了一间能住 30 天的房间。大多数客人只住 3 天就走，但房间不能转租，空着直到第 30 天。这就是静态分配的浪费：按最坏情况预分配，实际利用率很低。如果改成按天分配房间，客人走了立刻回收，酒店就能接待更多客人。

本章介绍两种 KV Cache 管理策略来消除这种浪费：**Paged Attention** 通过分块按需分配解决内部碎片问题，**Ring KV Cache** 通过环形缓冲区适配滑动窗口特性。

2 Paged Attention：按需分块

2.1 核心理念

Paged Attention 借鉴了操作系统的虚拟内存分页机制。它不再为每个请求预分配一整块连续的 KV Cache，而是将 KV Cache 划分为固定大小的 **Block**（块），从一个全局 **Block Pool**（块池）中按需分配。

每个请求维护一张 **Block Table**（块表），记录逻辑块号到物理块号的映射。逻辑上请求的 KV Cache 是连续的，物理上各块散布在 Block Pool 的不同位置。

```
Block Pool (物理):
[ B0 ][ B1 ][ B2 ][ B3 ][ B4 ][ B5 ][ B6 ][ B7 ] ...
 空闲  Req1  空闲  Req1   Req2  空闲  Req2   Req1

Block Table:
Req1: [ B1 ] [ B3 ] [ B7 ] ...   (逻辑块0→B1, 1→B3, 2→B7)
Req2: [ B4 ] [ B6 ] ...         (逻辑块0→B4, 1→B6)
```

请求需要更多 KV 时，从空闲列表取出一个物理块，在 Block Table 中追加映射。请求结束时，该请求占用的所有物理块回收到空闲列表，立即可供新请求使用。

2.2 Block Size 的权衡

Block Size P 是每个物理块包含的 token 数量。它直接影响碎片和开销：

1. **碎片**：每个请求最多浪费 $P - 1$ 个槽位（最后一块未填满）。 P 越小，浪费越少。
2. **Block Table**： P 越小，同一请求需要的块数越多，Block Table 越大，寻址开销越高。

3. **注意力计算**：块越细，注意力算子需要遍历的离散块越多，可能影响 GPU 并行效率。

直觉 P 就像酒店的房间大小。房间太大 ($P = 256$)，一个人住一间很浪费；房间太小 ($P = 1$)，虽然不浪费空间，但管理无数间房间的登记簿 (Block Table) 本身就成了负担。实际中取 $P = 16$ 是一个不错的折中。

2.3 功能正确版本

Paged Attention 的功能正确版本分四步：

1. **Block Pool + Block Table**：维护全局物理块池和每个请求的块表
2. **收集 KV Block**：注意力计算前，按 Block Table 把该请求的 KV 物理块收集到连续内存
3. **复用注意力**：在收集后的连续 KV 上执行标准注意力计算
4. **写回**：计算完成后，新产生的 KV 写回对应物理块

这个版本正确但效率不高，因为每次注意力计算都需要额外的收集和写回拷贝。

2.4 优化版本

优化版本让注意力算子直接读取离散的物理块，跳过收集步骤。注意力 kernel 按 block 遍历 KV，通过 Block Table 查询每个 block 的物理地址，直接从全局内存读取。这要求注意力 kernel 感知 block 结构，实现复杂度更高，但消除了中间拷贝。

Lab5 的实验框架使用功能正确版本即可满足正确性要求。优化版本是 vLLM 等工业级推理引擎的做法，需要自定义 CUDA kernel。

3 Ring KV Cache：滑动窗口的天然适配

3.1 滑动窗口的特性

Gemma4-12B 的大部分层使用滑动窗口注意力，窗口大小 $w = 1024$ 。这意味着位置 t 的 query 只关注 $\max(0, t - w + 1)$ 到 t 的 key，更早的 key 永远不会被访问。

如果用静态分配，这些永远不被访问的旧 KV 仍然占据显存。序列长度 2048 时，后半段有 1024 个 token 的 KV 完全无用，却仍然占着空间。

3.2 环形缓冲区

Ring KV Cache（环形 KV 缓存）用一个固定大小为 w 的环形缓冲区替代线性 KV Cache。位置 t 的 KV 写入槽位 $t \bmod w$ ，当 $t > w$ 时，新的 KV 覆盖最旧的 KV。

Ring KV Cache ($W=4$):

```
t=0: [K0][ ][ ][ ]
t=1: [K0][K1][ ][ ]
t=2: [K0][K1][K2][ ]
t=3: [K0][K1][K2][K3]
```

t=4: [K4][K1][K2][K3] K0 被覆盖 (不再需要)
t=5: [K4][K5][K2][K3] K1 被覆盖

位置 t 写入槽位 $t \bmod W$ ，读取时也通过 $t \bmod W$ 定位。由于窗口大小恰好为 w ，被覆盖的 KV 一定在窗口之外，不影响注意力范围。

直觉 想象一个只有 w 个格子的环形跑道。你一边跑一边在格子上写最新的记录，跑完一圈后新记录覆盖最旧的。因为注意力只看最近 w 步，被覆盖的旧记录本来就不会再看，覆盖它毫无损失。

3.3 显存从 $O(S)$ 降到 $O(\min(S, W))$

使用 Ring KV Cache 后，滑动窗口层的 KV Cache 大小从 $O(S)$ 降为 $O(\min(S, W))$ ：

$$\text{KV}_{\text{ring}} = \begin{cases} S & \text{if } S < W \\ W & \text{if } S > W \end{cases} \quad (3.1)$$

当序列长度远超窗口时 ($S > W$)，节省比例为 $\frac{S-W}{S}$ 。Ring KV Cache 不改变注意力的计算范围和结果，只是复用存储空间，因此无精度损失。

3.4 混合注意力层的 KV 因子

Gemma4-12B 采用混合注意力： N_g 个全局注意力层和 N_l 个滑动窗口层。全局层的 KV 随序列长度 S 线性增长，滑动窗口层的 KV 被限制在 W 以内。整个模型的 KV Cache 总量为：

$$\text{KV}_{\text{total}} = N_g S + N_l \min(S, W) \quad (3.2)$$

当 $S > W$ 时， N_l 个滑动窗口层的 KV 被压缩到 W ，只有 N_g 个全局层继续增长。由于 N_l 远大于 N_g (Gemma4-12B 中大部分层是滑动窗口层)，总 KV Cache 的增长速度被大幅减缓。

4 实战：算一算省了多少

4.1 Ring KV 的显存节省

例 取滑动窗口 $w = 1024$ ，序列长度 $S = 2048$ 。不使用 Ring KV 时，滑动窗口层的 KV Cache 占用与 S 成正比。使用 Ring KV 后，占用与 $\min(S, W)$ 成正比。

节省比例：

$$\frac{S - \min(S, W)}{S} = \frac{2048 - 1024}{2048} = \frac{1024}{2048} = 50\% \quad (4.1)$$

即滑动窗口层的 KV Cache 显存减半。如果序列更长，节省更多。取 $S = 4096$ ：

$$\frac{4096 - 1024}{4096} = \frac{3072}{4096} = 75\% \quad (4.2)$$

四分之三的 KV Cache 被节省。

4.2 Paged Attention 的碎片对比

例 取 block size $P = 16$, 3 个请求的实际长度分别为 100、500、1000 token。假设静态分配按最大长度 $S = 1000$ 预分配。

静态分配: 每个请求分配 $S = 1000$ 个槽位, 共 $3 \times 1000 = 3000$ 个槽位。实际使用 $100 + 500 + 1000 = 1600$ 个, 浪费 $3000 - 1600 = 1400$ 个, 浪费率 $\frac{1400}{3000} \approx 46.7\%$ 。

Paged Attention: 每个请求按实际长度分块, 最后一块未填满最多浪费 $P - 1 = 15$ 个槽位。

Req1 (长度 100): $\lceil \frac{100}{16} \rceil = 7$ 块, 使用 100, 浪费 $7 \times 16 - 100 = 12$ 。Req2 (长度 500): $\lceil \frac{500}{16} \rceil = 32$ 块, 使用 500, 浪费 $32 \times 16 - 500 = 12$ 。Req3 (长度 1000): $\lceil \frac{1000}{16} \rceil = 63$ 块, 使用 1000, 浪费 $63 \times 16 - 1000 = 8$ 。

总分配 $7 + 32 + 63 = 102$ 块 = 1632 槽位, 总浪费 $12 + 12 + 8 = 32$, 浪费率 $\frac{32}{1632} \approx 2.0\%$ 。

从 46.7% 降到 2.0%, 碎片几乎消除。

5 框架中的 KV Cache 实现

Lab5 的实验框架使用 LayerKVCache 来管理每一层的 KV Cache。其形状为 $[\text{max_batch}, \text{kv_heads}, \text{max_seq_len}, \text{head_dim}]$, 即按最大 batch 和最大序列长度预分配。

在 decode 阶段, 每生成一个 token, 框架通过 `index_copy_` 将新的 K 和 V 按绝对位置写入 KV Cache 对应槽位:

```
layer_kv_cache.k.index_copy_(
    dim=2, index=positions, source=new_k
)
layer_kv_cache.v.index_copy_(
    dim=2, index=positions, source=new_v
)
```

`dim=2` 指定序列维度, `index` 是当前 token 的绝对位置, `source` 是新计算的 K 或 V。
`index_copy_` 直接按索引覆写, 无需移动已有数据。

框架的 `LayerKVCache` 是静态分配版本。实现 *Paged Attention* 时, 需要将这个连续张量改为 *Block Pool* 加 *Block Table* 的结构。实现 *Ring KV Cache* 时, 只需在 `index_copy_` 时将绝对位置对 w 取模, 是最简单的优化。

6 本章你将学会

1. 解释静态分配 KV Cache 导致的内部碎片和显存浪费
2. 描述 Paged Attention 的 Block Pool 和 Block Table 机制
3. 分析 Block Size P 对碎片和寻址开销的权衡
4. 区分功能正确版本 (收集再计算) 和优化版本 (直接读离散块)
5. 解释 Ring KV Cache 如何用环形缓冲区适配滑动窗口

6. 推导混合注意力层的 KV 因子 $N_g S + N_l \min(S, W)$
7. 估算 Paged Attention 和 Ring KV 在具体场景下的显存节省比例
8. 理解框架中 LayerKVCache 的形状和 index_copy_ 写入方式

7 要点速查

要点	说明
静态分配浪费	按 $B \times S$ 预分配，短请求浪费严重
Paged Attention	Block Pool 按需分配，Block Table 记录映射
Block Size P	越小碎片越少，但 Block Table 更大
每请求最大浪费	$P - 1$ 个槽位
功能正确版	收集 KV Block 到连续内存再算注意力
优化版	注意力 kernel 直接读离散物理块
Ring KV Cache	位置 t 写入 $t \bmod W$ ，覆盖窗口外旧 KV
Ring KV 大小	$O(\min(S, W))$ ，无精度损失
混合层 KV 因子	$N_g S + N_l \min(S, W)$
LayerKVCache 形状	<code>[max_batch, kv_heads, max_seq_len, head_dim]</code>

8 小结

KV Cache 的显存管理是提升推理吞吐量的关键杠杆。Paged Attention 通过分页机制将静态分配的内存碎片从近 50% 降到约 2%，让显存利用率紧贴实际需求。Ring KV Cache 则针对滑动窗口注意力的特性，用固定大小的环形缓冲区将 KV Cache 从 $O(S)$ 压缩到 $O(\min(S, W))$ ，序列越长节省越多。两者都不改变注意力的数学结果，纯粹是存储管理的优化。

在 Gemma4-12B 的混合注意力架构中，全局层和滑动窗口层的 KV 行为不同：全局层的 KV 随序列增长不受限制，滑动窗口层的 KV 被窗口封顶。总 KV Cache 由 $N_g S + N_l \min(S, W)$ 决定，由于滑动窗口层占多数，整体增长被大幅减缓。理解这些机制后，下一步我们将注意力转向计算本身的优化：Flash Attention 如何在不物化注意力矩阵的前提下完成精确注意力计算。

讲义基于 HPC Lab5 实验指导编写